DSP GTU SUPER 40 QUE ANS

UNIT 1. (9%)

Q1. List types of Data Structures.

Data structures are broadly classified into two main categories: Primitive and Non-Primitive data structures.

1. Primitive Data Structures
These are basic data types that are directly supported by programming languages.
They store simple and small values.

Examples:

Integer
Float
Character
Boolean
String

2. Non-Primitive Data Structures
These data structures are more complex and used to store large collections of data.
They are further divided into two types:

A. Linear Data Structures
Data elements are arranged in a sequence, one after another.

Examples:

Array
Linked List
Stack
Queue

B. Non-Linear Data Structures
Data elements are connected in a hierarchical or network structure rather than a single line.

Examples:
Tree
Binary Search Tree
Graph
Heap

Q2. Differentiate between Linear and Non-Linear Data Structures.

Data structures can be divided into linear and non-linear based on how data elements are arranged.

Linear Data Structures

In a linear data structure, data elements are stored one after another in a sequential manner. Each element has a unique predecessor and successor, except the first and last element.

Examples: Array, Linked List, Stack, Queue.

Features:

Easy to implement and traverse.
Memory is arranged in a continuous or logical sequence.
Suitable for simple data storage.

Non-Linear Data Structures

In a non-linear data structure, elements are not stored in sequence.
They are arranged in a hierarchical or network manner where one element can be connected to multiple elements.

Examples: Tree, Binary Tree, Graph, Heap.

Features:

More complex structure compared to linear DS.
Suitable for representing relationships like parent–child.
Traversal is more complex.

Q3. Explain Primitive and Non-Primitive Data Structures.

Data structures are divided into two major categories: Primitive and Non-Primitive. Both are used to store data, but they differ in structure and complexity.

1. Primitive Data Structures

Primitive data structures are the basic, built-in data types provided by programming languages. They store simple and single values.

Examples:

Integer – stores whole numbers
Float – stores decimal numbers

Character – stores single characters
Boolean – stores TRUE or FALSE
String – stores sequence of characters

Characteristics:

Simple and easy to use
Fixed size
Directly operated by machine instructions

2. Non-Primitive Data Structures
Non-primitive data structures are more complex and used to store large collections of data.
They can be built using primitive data types.

They are further divided into Linear and Non-Linear structures:

A. Linear Data Structures
Data is arranged in sequential order.
Examples: Array, Linked List, Stack, Queue

B. Non-Linear Data Structures
Data is arranged hierarchically or in network form.
Examples: Tree, Graph, Heap

Characteristics:

Can store multiple values
Flexible size
Used for complex data organization

Q4. Define the following terms: Time Complexity and Space Complexity.
1. Time Complexity

Time Complexity is a measure that tells how much time an algorithm takes to complete its execution based on the size of the input.
It helps us understand the efficiency of an algorithm.

It is expressed using asymptotic notations, such as:

O(n) → Big O (worst case)
Ω(n) → Omega (best case)
Θ(n) → Theta (average/exact case)

Example:

Linear search takes O(n) time because it may check each element once.

2. Space Complexity

Space Complexity defines how much memory an algorithm requires during its execution.
This includes memory used by variables, arrays, objects, recursion stack, etc.

Space complexity is also expressed using notations like O(1), O(n), O(n^2).

Example:
Storing an array of size n requires O(n) space.

Q5. Define Best Case, Worst Case and Average Case for Time Complexity.
When analyzing an algorithm, we study how much time it takes in different situations. These situations are represented as best case, worst case, and average case.

1. Best Case

The best case is the situation where the algorithm takes the minimum possible time to complete its execution.
It happens when the input is in the most favorable condition.
Represented using Ω (Omega) notation.

Example:
In linear search, if the element is found at the first position, the time complexity is Ω(1).

2. Worst Case

The worst case is the situation where the algorithm takes the maximum possible time to complete its execution.
It represents the least favorable or heaviest input condition.
Represented using O (Big O) notation.

Example:
In linear search, if the element is found at the last position or not present, the time complexity becomes O(n).

3. Average Case

The average case shows the expected time taken by the algorithm for a typical or random input.
It assumes all input cases occur with equal probability.
Represented using Θ (Theta) notation.

Example:

For linear search, average case is when the element appears somewhere in the middle, giving Θ(n) time.

UNIT 2. (17%)

Q.6. Explain Python Specific Data Structures: List, Tuple, and Dictionary.

Python provides several built-in data structures to store and manage data efficiently. Three commonly used structures are List, Tuple, and Dictionary.

1. List

A list is an ordered collection of elements.
It is mutable, which means elements can be changed, added, or removed.
List allow duplicate elements.

Example:

```
my_list = [10, 20, 30, 40]
my_list.append(50)  # Adding element
```

2. Tuple

A tuple is an ordered collection of elements.
It is immutable, which means elements cannot be changed after creation.
Tuples allow duplicate elements.

Example:

```
my_tuple = (10, 20, 30)
# my_tuple[0] = 50  # This will cause an error
```

3. Dictionary

A dictionary stores key–value pairs.
It is unordered and mutable.
Keys must be unique, but values can be duplicate.

Example:

```
my_dict = {"name": "Alice", "age": 20}
my_dict["age"] = 21  # Update value
```

Q7. Explain Set Data Structure in Python.

A set in Python is a collection of unique and unordered elements. It is used to store multiple items without duplicates.

Key Features of Set

1. Unordered – Elements do not have a specific index.
2. Unique Elements – Duplicate values are automatically removed.
3. Mutable – You can add or remove elements.
4. No Indexing – You cannot access elements using an index.

Creating a Set

```
my_set = {10, 20, 30, 40}
print(my_set)  # Output: {10, 20, 30, 40}
```

Basic Operations on Sets

Operation	**Description**	Example
add()	**Add a new element**	my_set.add(50)
remove()	**Remove an element**	my_set.remove(20)
union()	**Combine two sets**	A.union(B)
intersection()	**Common elements**	A.intersection(B)
difference()	**Elements in one set not in other**	A.difference(B)

Q8. Define Tuple in Python and Explain Its Operations.

A tuple in Python is an ordered, immutable collection of elements. Once created, the elements of a tuple cannot be modified (no addition, deletion, or change of elements).

Key Features of Tuple

1. Ordered – Elements maintain their order.
2. Immutable – Cannot change elements after creation.
3. Allows duplicates – Same element can appear multiple times.
4. Indexed – Elements can be accessed using an index.

Creating a Tuple

```
my_tuple = (10, 20, 30, 40)
print(my_tuple[0])  # Output: 10
```

Common Operations on Tuples

Operation	Description	Example
Indexing	Access elements by index	my_tuple[2] → 30
Slicing	Extract subset	my_tuple[1:3] → (20, 30)
Count()	Count occurrence of an element	my_tuple.count(20) → 1
Index()	Find index of element	my_tuple.index(30) → 2
Concatenation	Join two tuples	(1,2) + (3,4) → (1,2,3,4)
Repetition	Repeat elements	(1,2)*2 → (1,2,1,2)

Q9. Compare List and Tuple in Python.

List and Tuple are both ordered collections in Python, but they have key differences.

Feature	List	Tuple
Mutability	Mutable (can change elements)	Immutable (cannot change elements)
Syntax	[] (square brackets)	() (parentheses)
Performance	Slower	Faster
Memory	Consumes more memory	Consumes less memory
Use Case	Suitable for dynamic data that changes frequently	Suitable for fixed data that should not change
Methods	Many built-in methods like append(), remove(), sort()	Limited methods like count() and index()

Q10. Differentiate Between Array and List.

Although both arrays and lists are used to store collections of elements, they have some important differences:

Feature	Array	List
Size	Fixed size (defined at creation)	Dynamic size (can grow or shrink)
Data Type	Stores elements of the same data type	Can store elements of different data types
Memory	Contiguous memory allocation	Flexible memory allocation
Performance	Faster for operations like access or iteration	Slightly slower due to dynamic resizing
Syntax in Python	Use array module: array.array('i', [1,2,3])	Use square brackets: [1, "hello", 3.5]
Use Case	Efficient for large numeric data	Useful for general-purpose collection of items

UNIT 3. (18%)

Q11. Explain Different Concepts of Object-Oriented Programming (OOP).

Object-Oriented Programming (OOP) is a programming paradigm that organizes code using objects and classes. The main concepts of OOP are:

1. Class

A class is a blueprint or template for creating objects.
It defines attributes (data) and methods (functions) that objects of the class can have.

Example: class Car: pass

2. Object

An object is an instance of a class.
It represents a real-world entity with data and behavior.

Example: my_car = Car()

3. Encapsulation

Encapsulation means hiding internal details and providing access only through methods.
It protects data from direct modification.

Example: Using private variables with getter and setter methods.

4. Inheritance

Inheritance allows a class (child) to reuse properties and methods of another class (parent).
Helps in code reusability.

Example: class ElectricCar(Car): pass

5. Polymorphism

Polymorphism allows objects to take many forms.
The same method name can behave differently depending on the object.

Example: Method overriding in child class.

6. Abstraction

Abstraction hides complex implementation details and shows only essential features.
Achieved using abstract classes or interfaces.

Q12. Explain the Concept of Class and Object with the Help of Python Program.

1. Class

A class is a blueprint or template used to create objects.
It defines attributes (data) and methods (functions) that its objects will have.

Syntax in Python:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
    def display(self):
        print(f"Car Brand: {self.brand}, Model: {self.model}")
```

2. Object

An object is an instance of a class.
It represents a real-world entity and contains data and behavior defined in the class.

Creating Objects:

```
my_car = Car("Toyota", "Corolla")
my_car.display()
# Output: Car Brand: Toyota, Model: Corolla
```

Q13. Define Constructor and Discuss Different Types of Constructors with Python Example.

1. Constructor

A constructor is a special method in a class used to initialize the attributes of an object when it is created.
In Python, the constructor is defined using the __init__() method.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
s1 = Student("Alice", 20)
print(s1.name, s1.age)  # Output: Alice 20
```

2. Types of Constructors in Python

A. Default Constructor

A constructor without parameters.
Python provides a default constructor if none is defined.

Example:

```
class Car:
    def __init__(self):
        self.brand = "Unknown"
c1 = Car()
print(c1.brand)  # Output: Unknown
```

B. Parameterized Constructor

A constructor that accepts arguments to initialize object attributes.

Example:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
c1 = Car("Toyota", "Corolla")
print(c1.brand, c1.model)  # Output: Toyota Corolla
```

C. Default + Parameterized

You can combine both using default values.

Example:

```
class Car:
    def __init__(self, brand="Toyota", model="Yaris"):
        self.brand = brand
        self.model = model
c1 = Car()
c2 = Car("Honda", "Civic")
print(c1.brand, c1.model)  # Output: Toyota Yaris
print(c2.brand, c2.model)  # Output: Honda Civic
```

Q14. Explain Various Types of Methods in OOP (Python).

In Python Object-Oriented Programming (OOP), a method is a function defined inside a class to perform operations using class attributes or objects. There are mainly three types of methods:

1. Instance Method

The most common type of method.
Operates on instance variables of the object.
Requires self as the first parameter to access object attributes.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")
s1 = Student("Alice", 20)
s1.display()  # Output: Name: Alice, Age: 20
```

2. Class Method

Defined with the @classmethod decorator.
Operates on class variables, not instance variables.
Takes cls as the first parameter.

Example:

```
class Student:
    school = "ABC High School"
    @classmethod
    def show_school(cls):
        print(f"School: {cls.school}")
Student.show_school()  # Output: School: ABC High School
```

3. Static Method

Defined with the @staticmethod decorator.
Does not require self or cls.
Cannot access instance or class variables directly.
Used for utility functions related to the class.

Example:

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
print(Math.add(10, 5))  # Output: 15
```

Q15. Differentiate Between Class Method and Static Method

Difference Table

Point	Class Method	Static Method
Decorator	@classmethod	@staticmethod
First Parameter	Takes cls (refers to class)	Takes no default parameter
Access to Class Variables	Yes, can access and modify class variables	No, cannot access or modify class or instance variables directly
Relation with Class	Related to the class, not object	Utility function; independent of class and object
How to Call	Using class or object	Using class or object
Use Case	When you need to work with class-level data	When you need a helper function inside class

Example of Class Method

```
class Student:
    school = "ABC School"

    @classmethod
    def show_school(cls):
        print("School Name:", cls.school)
Student.show_school()
```

Example of Static Method

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
print(Math.add(10, 5))
```

Q16. Explain Types of Inheritance

Inheritance is a feature of Object-Oriented Programming where one class (child class) can acquire the properties and methods of another class (parent class).

Python supports five main types of inheritance.

1. Single Inheritance

A child class inherits from one parent class.

Example:

```
class Parent:
    def show(self):
        print("Parent Class")
class Child(Parent):
    pass
```

2. Multiple Inheritance

A child class inherits from two or more parent classes.

Example:

```
class A:
    pass
class B:
    pass
class C(A, B):
    pass
```

3. Multilevel Inheritance

A class is derived from another derived class (a chain of inheritance).

Example:

```
class A:
    pass
class B(A):
    pass
class C(B):
    pass
```

4. Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

Example:

```
class Parent:
    pass
class Child1(Parent):
    pass
class Child2(Parent):
    pass
```

5. Hybrid Inheritance

A combination of multiple types of inheritance (single + multiple + multilevel etc.)

Q17. Explain Polymorphism with Suitable Example

Polymorphism means one name, many forms.
In Python, polymorphism allows the same function or operator to behave differently based on the object or data type.

Types of Polymorphism in Python

1. Method Overriding (Runtime Polymorphism)
2. Method Overloading (not directly supported but can be simulated)
3. Operator Overloading

1. Method Overriding Example

A child class can provide a new version of a method already defined in the parent class.

```
class Animal:
    def sound(self):
        print("Animal makes sound")
class Dog(Animal):
    def sound(self):
        print("Dog barks")
a = Animal()
d = Dog()
a.sound()   # Output: Animal makes sound
d.sound()   # Output: Dog barks
```

2. Polymorphism with Functions

A single function can take different types of objects and call their respective methods.

```
class Cat:
    def sound(self):
        print("Meow")
class Cow:
    def sound(self):
        print("Moo")
def animal_sound(animal):
    animal.sound()
animal_sound(Cat())   # Meow
animal_sound(Cow())   # Moo
```

3. Operator Overloading Example

The + operator works differently on strings and numbers.

```
print(5 + 10)       # 15 (addition)
print("Hi " + "All") # Hi All (string concatenation)
```

Q18. Define Abstraction. Write a Python program to understand the concept of Abstract Class.

Definition of Abstraction

Abstraction is an OOP concept that hides the internal implementation details and shows only the essential features to the user.

Example:

When you drive a car, you use the steering, brakes, and accelerator, but you do not know the internal engine working.

This is abstraction.

Python provides abstraction using the abc (Abstract Base Class) module.

Abstract Class

An abstract class is a class that cannot be instantiated.
It contains one or more abstract methods.
An abstract method is a method declared but not implemented in the abstract class.

To create abstraction, we use:

from abc import ABC, abstractmethod
Python Program Demonstrating Abstract Class

```
from abc import ABC, abstractmethod
class Shape(ABC):       # Abstract Class
    @abstractmethod
    def area(self):
        pass
class Rectangle(Shape):  # Child Class
    def area(self):
        return "Area = length × width"
class Circle(Shape):     # Child Class
    def area(self):
        return "Area = π × radius²"
# Creating objects of child classes
r = Rectangle()
c = Circle()
print(r.area())
print(c.area())
```

UNIT 4. (20%)

Q19. Define Stack. Explain PUSH & POP Operation with Example. Write an Algorithm for PUSH and POP.

What is a Stack?

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle.
This means the element inserted last will be removed first.

Examples:

Stack of books
Stack of plates

1. PUSH Operation
PUSH means inserting (adding) an element into the stack.

Conditions:

If the stack is full → Overflow
Otherwise → Insert element at the top

2. POP Operation
POP means removing an element from the stack.

Conditions:

If the stack is empty → Underflow
Otherwise → Remove and return element from the top

Example of PUSH and POP

Stack: [10, 20, 30]
Top → 30

PUSH(40) → [10, 20, 30, 40]
POP() → removes 40 → [10, 20, 30]

Algorithm for PUSH Operation

PUSH(stack, item):

```
1. IF top == max_size - 1 THEN
2.     PRINT "Stack Overflow"
3. ELSE
4.     top = top + 1
5.     stack[top] = item
6. END IF
```

Algorithm for POP Operation

POP(stack):

```
1. IF top == -1 THEN
2.     PRINT "Stack Underflow"
3. ELSE
4.     item = stack[top]
5.     top = top - 1
6.     RETURN item
7. END IF
```

Q20. Explain Applications of Stack
A stack is a linear data structure that works on LIFO (Last In, First Out) principle.

It is widely used in many areas of computer science.

Applications of Stack

1. Function Call Management

When a function is called, its local variables, return address and data are stored on the stack memory.
When the function ends, data is popped out.
Used in recursion.

2. Expression Evaluation

Stacks are used to evaluate:

Infix
Prefix
Postfix
expressions.

Example:
Postfix evaluation uses stack to compute expressions like 23*54*+.

3. Expression Conversion

Stack is used to convert:

Infix → Postfix
Infix → Prefix
Prefix → Infix

4. Undo/Redo System

Applications like:

MS Word
Photoshop
Browsers
use stacks to undo and redo previous actions.

5. Backtracking

Stack helps in reversing steps when trying alternate paths.

Used in:

Maze solving
Game moves
Searching algorithms

6. Browser History

Web browsers store visited pages in a stack.

Current page → top
Back button → pop
Forward → push again

7. Parentheses Balancing

Used to check if parentheses ()[]{} are balanced using stack.

8. Reversing Data

Stack can reverse:

Strings
Numbers
Lists

Q21. Explain Implementation of Queue Using List

A queue is a linear data structure that follows the FIFO (First In, First Out) principle.
In Python, a queue can be implemented easily using a list.

Queue Operations Using List

The main operations are:

1. Enqueue (Insert an element)
2. Dequeue (Remove an element)
3. Front (First element of queue)
4. Rear (Last element of queue)
5. Display (Show all elements)

1. Enqueue Operation (Insert)

Adds an element at the end of the list.

Implemented using append().

queue.append(item)

2. Dequeue Operation (Delete)

Removes an element from the front of the list.
Implemented using pop(0).

queue.pop(0)

⭐ Python Program: Queue Implementation Using List

```
queue = []     # empty queue

# Enqueue elements
queue.append(10)
queue.append(20)
queue.append(30)
print("Queue after enqueue:", queue)

# Dequeue
removed = queue.pop(0)
print("Removed element:", removed)

print("Queue after dequeue:", queue)
```

⭐ Output

Queue after enqueue: [10, 20, 30]
Removed element: 10
Queue after dequeue: [20, 30]

Q22. Define Circular Queue. Explain INSERT and DELETE Operations of Circular Queue with Diagrams

What is a Circular Queue?

A circular queue is a linear data structure in which the last position is connected back to the first position, forming a circle.
It is used to efficiently use memory and avoid wasted space of a simple queue.

⭐ Diagram of Circular Queue

Indexes: 0 1 2 3 4
Queue: [10][20][30][ ][ ]
front → 0
rear → 2

If rear moves beyond index 4, it goes back to 0.

⭐ INSERT Operation (Enqueue)

Conditions

If (rear + 1) % size == front → Queue is Full
Else insert at rear = (rear + 1) % size

Steps

1. Check if queue is full
2. Move rear to next circular position
3. Insert the element
4. If queue was empty, set front = 0

Diagram for INSERT

Before insert:

[10][20][30][ ][ ]
front = 0
rear = 2

Insert 40 →

[10][20][30][40][ ]
front = 0
rear = 3

⭐ DELETE Operation (Dequeue)

Conditions

If front == -1 → Queue is Empty

Else remove the element at front

Steps

1. Check if queue is empty
2. Remove element from front
3. Move front to (front + 1) % size
4. If after deletion front == rear, set both to -1

Diagram for DELETE

Before delete:

[10][20][30][40][ ]
front = 0
rear = 3

Delete 10 →

[ ][20][30][40][ ]
front = 1
rear = 3

Q23. Differentiate Between Simple Queue and Circular Queue

Difference Between Simple Queue and Circular Queue

Simple Queue Circular Queue

1. Linear structure where elements are arranged in a straight line.
1. Forms a circular structure where last position connects to the first.
2. When rear reaches the end, no more insertions even if space is available at the beginning.
2. Rear automatically moves to the first position using modulo (%) operation.
3. Inefficient use of memory because empty spaces at the front cannot be reused.
3. Efficient memory use because empty spaces are reused.
4. Overflow occurs when rear == size-1.
4. Overflow occurs when (rear + 1) % size == front.
5. Deletion moves front forward linearly.
5. Deletion moves front using circular increment: (front + 1) % size.
6. Suitable for simple tasks with fewer operations.
6. Suitable for applications needing continuous use like CPU scheduling.
7. Implementation is simpler.
7. Implementation is slightly complex.

⭐ Diagram (Simple Queue)

[10][20][30][ ][ ]
front → 0
rear → 2
(No more reuse of empty space at front)

⭐ Diagram (Circular Queue)

[ ][20][30][40][50]
front → 1
rear → 4
(rear wraps to front using modulo)

UNIT 5. (20%)

Q24. Explain the difference between Authentication and Authorization.
Authentication and authorization are two important security processes, but they perform different functions.

Authentication is the process of verifying the identity of a user. It checks “Who are you?” Common methods include passwords, OTP, biometrics, and ID cards. If the credentials are correct, the user is successfully authenticated.

Authorization happens after authentication. It decides “What are you allowed to do?” It checks the permissions and access levels of the authenticated user. For example, a user may log in (authentication) but may not have permission to edit files (authorization).

In simple terms:

Authentication = Identity verification
Authorization = Permission checking

Example:
When you log into Gmail with a password, the system authenticates you. After login, you can access only your emails, not someone else’s — this is authorization.

Q25. Explain Operations for Inserting a Node at Beginning and End of a Singly Linked List
A singly linked list is a linear data structure where each node contains data and a pointer to the next node.

1. Insert at Beginning (Head)

Steps:

1. Create a new node.
2. Set new node's next pointer to point to the current head.
3. Update head to the new node.

Algorithm:

INSERT_AT_BEGINNING(data):
1. new_node = createNode(data)
2. new_node.next = head
3. head = new_node

Example:

Original List: 10 → 20 → 30
Insert 5 at beginning
New List: 5 → 10 → 20 → 30

2. Insert at End (Tail)

Steps:

1. Create a new node.
2. Traverse the list to reach the last node.
3. Set last node's next to new node.
4. Set new node's next to NULL.

Algorithm:

INSERT_AT_END(data):
1. new_node = createNode(data)
2. if head is NULL:
    head = new_node
    return
3. temp = head
4. while temp.next != NULL:
    temp = temp.next
5. temp.next = new_node
6. new_node.next = NULL

Example:

Original List: 10 → 20 → 30
Insert 40 at end
New List: 10 → 20 → 30 → 40

Q26. Explain Operations for Deleting a Node from Beginning and End of a Singly Linked List

A singly linked list is a linear data structure where each node contains data and a pointer to the next node. Deletion can be performed at the beginning or end of the list.

1. Delete from Beginning (Head)

Steps:

1. Check if the list is empty (head == NULL). If yes, print Underflow.
2. Save the current head node in a temporary variable.
3. Move head to the next node (head = head.next).
4. Delete the temporary node.

Algorithm:

```
DELETE_AT_BEGINNING():
1. if head == NULL:
       PRINT "List is empty"
       RETURN
2. temp = head
3. head = head.next
4. DELETE temp
```

Example:

Original List: 10 → 20 → 30
Delete from beginning
New List: 20 → 30

2. Delete from End (Tail)

Steps:

1. Check if the list is empty (head == NULL). If yes, print Underflow.
2. If only one node exists, delete it and set head = NULL.

3. Traverse the list to reach the second last node.
4. Delete the last node and set second last node's next = NULL.

Algorithm:

```
DELETE_AT_END():
1. if head == NULL:
      PRINT "List is empty"
      RETURN
2. if head.next == NULL:
      DELETE head
      head = NULL
      RETURN
3. temp = head
4. while temp.next.next != NULL:
      temp = temp.next
5. DELETE temp.next
6. temp.next = NULL
```

Example:

Original List: 10 → 20 → 30
Delete from end
New List: 10 → 20

Q27. Explain Circular Linked List with Its Advantages

What is a Circular Linked List?

A circular linked list (CLL) is a linked list in which the last node points back to the first node instead of pointing to NULL.
It can be singly or doubly linked.

Types:

1. Singly Circular Linked List (SCLL) – last node points to the first node.
2. Doubly Circular Linked List (DCLL) – last node points to the first node, and first node's previous pointer points to last node.

Diagram of Singly Circular Linked List

head → 10 → 20 → 30 → 40 → head

The next pointer of the last node (40) points to the head (10), forming a circle.

Advantages of Circular Linked List

1. Efficient Memory Utilization

No need for a fixed-size array; nodes are dynamically allocated.

2. Easier to Traverse

Can traverse the list from any node repeatedly because it is circular.

3. Useful for Queue Implementation

Ideal for circular queues.

4. Continuous Operations

Ideal for applications requiring round-robin scheduling.

5. No Null Termination

Unlike singly linked lists, no NULL pointer at the end, so easier looping.

Q28. What is a Doubly Linked List? Explain with Advantages

Definition

A doubly linked list (DLL) is a type of linked list in which each node contains three fields:

1. Data – stores the value
2. Next – points to the next node
3. Previous – points to the previous node

This allows traversal in both forward and backward directions.

Diagram of Doubly Linked List

NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL

<-> indicates bidirectional links.

head points to the first node.

tail points to the last node.

Advantages of Doubly Linked List

1. Bidirectional Traversal

Can traverse the list forward and backward.

2. Easy Insertion and Deletion

Inserting/deleting a node at any position is easier compared to singly linked list.

3. Better for Complex Operations

Useful in navigation systems, undo/redo operations, and browser history.

4. Efficient

Can move backward without starting from the head.

Q29. Differentiate Between Circular Linked List and Singly Linked List

Feature	Singly Linked List (SLL)	Circular Linked List (CLL)
Structure	Last node points to NULL	Last node points back to the first node
Traversal	Can traverse only from head to last node	Can traverse continuously from any node
Memory	Uses less memory (only next pointer per node)	Slightly more memory if circular pointers are maintained
Insertion/Deletion	Standard at beginning, end, or middle	Insertion/deletion at beginning or end requires adjusting circular link
Use Cases	Simple data storage and linear operations	Ideal for round-robin scheduling, buffering, or continuous looping
End of List	Traversal stops at NULL	Traversal can continue indefinitely, back to the head

Q30. Differentiate Between Singly Linked List and Doubly Linked List

Feature	Singly Linked List (SLL)	Doubly Linked List (DLL)
Structure	Each node has data and next pointer	Each node has data, next pointer, and previous pointer
Traversal	Can traverse only forward	Can traverse both forward and backward
Memory Requirement	Less memory (only one pointer per node)	More memory (two pointers per node)

Insertion/Deletion Easy at beginning, middle, end but backward traversal is not possible Easier insertion/deletion at any position due to previous pointer
Complexity Simpler to implement Slightly complex due to extra pointer management
Use Cases Simple linear data storage Applications requiring bi-directional traversal, like undo/redo operations, browser history, navigation

Q31. Differentiate Between Circular Linked List and Doubly Linked List

Feature Circular Linked List (CLL) Doubly Linked List (DLL)

Structure Last node points back to the first node Each node has next and previous pointers; last node points to NULL (or in circular DLL, last points to first)
Traversal Can traverse continuously in forward direction (or both in circular doubly) Can traverse forward and backward
Memory Requirement Less memory in singly circular (one pointer per node) More memory (two pointers per node)
Insertion/Deletion Insertion/deletion needs pointer adjustment to maintain circular link Easier insertion/deletion anywhere using previous pointer
Use Cases Suitable for round-robin scheduling, buffering, circular queues Suitable for applications requiring bi-directional navigation, undo/redo, browser history

UNIT 6. (16%)

Q32.Define Search and Write an Algorithm for Linear Search Using List
What is Search?

Search is the process of finding the position or presence of a specific element (key) in a data structure like an array, list, or linked list.

1. Linear Search

Linear search is a sequential search method.
It checks each element of the list one by one until the desired element is found or the list ends.
Works on unsorted and sorted lists.

Algorithm: Linear Search

```
LINEAR_SEARCH(list, key):
1. For i = 0 to length(list) - 1:
2.     If list[i] == key:
3.         PRINT "Element found at index", i
4.         RETURN
5. PRINT "Element not found"
```

Example

List = [10, 20, 30, 40, 50]
Key = 30

Step-by-step:
Index 0 → 10 ≠ 30
Index 1 → 20 ≠ 30
Index 2 → 30 == 30 → Found at index 2

Q32. Explain Binary Search Algorithm with Example
What is Binary Search?

Binary Search is an efficient search technique used on sorted lists.
It repeatedly divides the list into two halves and compares the key with the middle element until the key is found or the list becomes empty.
Works only on sorted arrays or lists.
Time complexity is much better than linear search: O(log n).

Algorithm: Binary Search

```
BINARY_SEARCH(list, key):
1. low = 0
2. high = length(list) - 1
3. while low <= high:
4.     mid = (low + high) // 2
5.     if list[mid] == key:
6.         PRINT "Element found at index", mid
7.         RETURN
8.     else if list[mid] < key:
9.         low = mid + 1
10.    else:
11.        high = mid - 1
12. PRINT "Element not found"
```

Example

List = [10, 20, 30, 40, 50]
Key = 30

Step 1: mid = (0+4)//2 = 2 → list[2] = 30

Step 2: list[2] == key → Found at index 2

Q33. Differentiate Between Linear Search and Binary Search

Feature	Linear Search	Binary Search
Data Requirement	Works on unsorted or sorted lists	Works only on sorted lists
Search Method	Sequentially checks each element	Divides the list and compares with the middle element
Time Complexity	Best: O(1), Worst: O(n), Average: O(n/2)	Best: O(1), Worst: O(log n), Average: O(log n)
Efficiency	Less efficient for large lists	More efficient for large lists
Implementation	Simple and easy	Slightly complex due to divide-and-conquer
Memory Usage	No extra memory needed	Can be implemented recursively (uses stack memory)
Use Case	Small or unsorted data	Large sorted data

Q34. Implement Bubble Sort Algorithm

What is Bubble Sort?

Bubble Sort is a simple comparison-based sorting algorithm.
It repeatedly compares adjacent elements and swaps them if they are in the wrong order.
After each pass, the largest element “bubbles” to the end of the list.

Algorithm: Bubble Sort

```
BUBBLE_SORT(list):
1. n = length(list)
2. for i = 0 to n-1:
3.     for j = 0 to n-i-2:
4.         if list[j] > list[j+1]:
5.             swap list[j] and list[j+1]
```

Example

Original List: [37, 22, 64, 84, 58, 52, 11]

Pass 1: [22, 37, 64, 58, 52, 11, 84]
Pass 2: [22, 37, 58, 52, 11, 64, 84]
Pass 3: [22, 37, 52, 11, 58, 64, 84]
Pass 4: [22, 37, 11, 52, 58, 64, 84]
Pass 5: [22, 11, 37, 52, 58, 64, 84]

Pass 6: [11, 22, 37, 52, 58, 64, 84] → Sorted

Time Complexity

Best Case: O(n) (already sorted, optimized version)
Worst Case: O(n²) (reversed order)
Average Case: O(n²)

Q35. Write an Algorithm for Bubble Sort with Trace Example
What is Bubble Sort?

Bubble Sort is a comparison-based sorting algorithm.
It repeatedly compares adjacent elements and swaps them if needed.
After each pass, the largest unsorted element moves to its correct position.

Algorithm: Bubble Sort

```
BUBBLE_SORT(list):
1. n = length(list)
2. for i = 0 to n-1:
3.     for j = 0 to n-i-2:
4.         if list[j] > list[j+1]:
5.             swap list[j] and list[j+1]
```

Trace Example

Data: 37, 22, 64, 84, 58, 52, 11

Step-by-Step Sorting:

Pass 1:

Compare 37 & 22 → swap → 22, 37, 64, 84, 58, 52, 11

Compare 37 & 64 → no swap

Compare 64 & 84 → no swap

Compare 84 & 58 → swap → 22, 37, 64, 58, 84, 52, 11

Compare 84 & 52 → swap → 22, 37, 64, 58, 52, 84, 11

Compare 84 & 11 → swap → 22, 37, 64, 58, 52, 11, 84

Pass 2:

Compare 22 & 37 → no swap

Compare 37 & 64 → no swap

Compare 64 & 58 → swap → 22, 37, 58, 64, 52, 11, 84

Compare 64 & 52 → swap → 22, 37, 58, 52, 64, 11, 84

Compare 64 & 11 → swap → 22, 37, 58, 52, 11, 64, 84

Pass 3:

Compare 22 & 37 → no swap

Compare 37 & 58 → no swap

Compare 58 & 52 → swap → 22, 37, 52, 58, 11, 64, 84

Compare 58 & 11 → swap → 22, 37, 52, 11, 58, 64, 84

Pass 4:

Compare 22 & 37 → no swap

Compare 37 & 52 → no swap

Compare 52 & 11 → swap → 22, 37, 11, 52, 58, 64, 84

Pass 5:

Compare 22 & 37 → no swap

Compare 37 & 11 → swap → 22, 11, 37, 52, 58, 64, 84

Pass 6:

Compare 22 & 11 → swap → 11, 22, 37, 52, 58, 64, 84

Sorted List: 11, 22, 37, 52, 58, 64, 84

Q36. Write an Algorithm for Selection Sort with Trace Example

What is Selection Sort?

Selection Sort is a comparison-based sorting algorithm.
It divides the list into sorted and unsorted parts.
Repeatedly selects the minimum element from unsorted part and moves it to the sorted part.

Algorithm: Selection Sort

```
SELECTION_SORT(list):
1. n = length(list)
2. for i = 0 to n-2:
3.     min_index = i
4.     for j = i+1 to n-1:
5.         if list[j] < list[min_index]:
6.             min_index = j
7.     swap list[i] and list[min_index]
```

Trace Example

Data: 13, 2, 6, 54, 18, 42, 11

Step 1: Find minimum from entire list → 2 → swap with 13
List: 2, 13, 6, 54, 18, 42, 11

Step 2: Find minimum from 13, 6, 54, 18, 42, 11 → 6 → swap with 13
List: 2, 6, 13, 54, 18, 42, 11

Step 3: Find minimum from 13, 54, 18, 42, 11 → 11 → swap with 13
List: 2, 6, 11, 54, 18, 42, 13

Step 4: Minimum from 54, 18, 42, 13 → 13 → swap with 54
List: 2, 6, 11, 13, 18, 42, 54

Step 5: Minimum from 18, 42, 54 → 18 → already in place
Step 6: Minimum from 42, 54 → 42 → already in place

Sorted List: 2, 6, 11, 13, 18, 42, 54

Q37. Explain Insertion Sort with Trace Example

What is Insertion Sort?

Insertion Sort is a comparison-based sorting algorithm.
It builds the sorted list one element at a time by inserting each element at its correct position.
Works well for small datasets and nearly sorted data.

Algorithm: Insertion Sort

```
INSERTION_SORT(list):
1. n = length(list)
2. for i = 1 to n-1:
3.     key = list[i]
4.     j = i - 1
5.     while j >= 0 and list[j] > key:
6.         list[j + 1] = list[j]
7.         j = j - 1
8.     list[j + 1] = key
```

Trace Example

Data: 25, 15, 30, 9, 99, 20, 26

Step-by-Step Sorting:

Step 1: i = 1, key = 15
Compare with 25 → 25 > 15 → move 25 → insert 15
List: 15, 25, 30, 9, 99, 20, 26

Step 2: i = 2, key = 30
Compare with 25 → 25 < 30 → insert 30
List: 15, 25, 30, 9, 99, 20, 26

Step 3: i = 3, key = 9
Compare with 30 → move 30
Compare with 25 → move 25
Compare with 15 → move 15 → insert 9
List: 9, 15, 25, 30, 99, 20, 26

Step 4: i = 4, key = 99

Compare with 30 → 30 < 99 → insert 99
List: 9, 15, 25, 30, 99, 20, 26

Step 5: i = 5, key = 20
Compare with 99 → move 99
Compare with 30 → move 30
Compare with 25 → move 25
Insert 20
List: 9, 15, 20, 25, 30, 99, 26

Step 6: i = 6, key = 26
Compare with 99 → move 99
Compare with 30 → move 30
Insert 26
List: 9, 15, 20, 25, 26, 30, 99

Sorted List: 9, 15, 20, 25, 26, 30, 99

Q38. Explain Merge Sort with Suitable Example
What is Merge Sort?

Merge Sort is a divide-and-conquer sorting algorithm.
It divides the list into halves, sorts each half, and merges them to produce a sorted list.
Works efficiently for large datasets.

Algorithm: Merge Sort

```
MERGE_SORT(list):
1. if length(list) > 1:
2.     mid = length(list) // 2
3.     left_half = list[0:mid]
4.     right_half = list[mid:]
5.     MERGE_SORT(left_half)
6.     MERGE_SORT(right_half)
7.     merge left_half and right_half into list
```

Merge Function:

```
MERGE(left, right, list):
1. i = j = k = 0
2. while i < length(left) and j < length(right):
3.     if left[i] <= right[j]:
4.         list[k] = left[i]
```

```
5.          i += 1
6.      else:
7.          list[k] = right[j]
8.          j += 1
9.      k += 1
10. Copy remaining elements of left (if any)
11. Copy remaining elements of right (if any)
```

Example

Data: 38, 27, 43, 3, 9, 82, 10

Step 1: Divide

Left = [38, 27, 43, 3]

Right = [9, 82, 10]

Step 2: Divide again

Left → [38, 27], [43, 3]

Right → [9, 82], [10]

Step 3: Sort each small list

[38, 27] → [27, 38]

[43, 3] → [3, 43]

[9, 82] → [9, 82]

[10] → [10]

Step 4: Merge

Left → [27, 38, 3, 43] → merge → [3, 27, 38, 43]

Right → [9, 82, 10] → merge → [9, 10, 82]

Step 5: Merge Left & Right → [3, 9, 10, 27, 38, 43, 82] ✅ Sorted List

Q39. Applications of Trees and Converting a General Tree into a Binary Search Tree (BST)
Applications of Trees

Trees are hierarchical data structures and widely used in computer science:

1. Hierarchical Data Representation

Example: File system directories, organization charts, XML/HTML documents.

2. Binary Search Tree (BST) for Searching

Provides efficient search, insertion, and deletion in O(log n) time for balanced trees.

3. Expression Trees

Used in compilers to represent arithmetic expressions.

4. Priority Queues / Heaps

Implemented using binary heaps for efficient extraction of minimum/maximum.

5. Routing Tables

Trees are used in network routing and IP address lookups.

6. Decision Making

Decision Trees in AI/ML for classification and prediction.

Converting General Tree into Binary Search Tree (BST)

Steps:

1. Traverse the General Tree

Perform in-order or level-order traversal to collect all nodes in a list.

2. Sort the Elements

Sort the collected node values in ascending order.

3. Construct BST

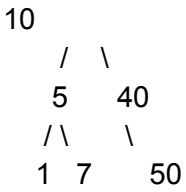
Take the middle element as the root.

Recursively take middle elements of left and right halves as left and right subtrees.

Example

General Tree Elements: 10, 5, 1, 7, 40, 50

Step 1: Collect → [10, 5, 1, 7, 40, 50]
Step 2: Sort → [1, 5, 7, 10, 40, 50]
Step 3: Build BST



Q40. Applications of Binary Tree

What is a Binary Tree?

A binary tree is a hierarchical data structure in which each node has at most two children, referred to as left child and right child.

Applications of Binary Tree

1. Hierarchical Data Representation

Binary trees represent hierarchical structures like organization charts or file systems.

2. Binary Search Tree (BST)

Efficient searching, insertion, and deletion of data in O(log n) time for balanced BSTs.

3. Expression Trees

Used in compilers to evaluate arithmetic expressions.
Internal nodes store operators; leaf nodes store operands.

4. Priority Queues / Heaps

Implement binary heaps for managing minimum or maximum values efficiently.

5. Sorting and Searching

Binary trees can be used in algorithms like tree sort.

6. Routing Tables and Decision Making

Binary trees help in network routing and decision trees in AI/ML.

7. Huffman Encoding

Binary trees are used in data compression algorithms like Huffman coding.

Q41. Define In-degree and Leaf Node in Binary Tree

Key Terms

1. Binary Tree

A hierarchical data structure where each node has at most two children, called left child and right child.

2. Level Number

Level of a node is the number of edges from the root node to that node.
Root node is at level 0 (or 1, depending on convention).

3. Leaf Node

A node with no children (both left and right pointers are NULL).
Example: In the tree below, 4, 5, and 6 are leaf nodes.



4. In-degree

Number of incoming edges to a node.

In a binary tree:

Root node → in-degree = 0
Other nodes → in-degree = 1

5. Out-degree

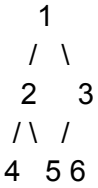
Number of children a node has.
For binary trees: maximum out-degree = 2.

Q42. Define Complete Binary Tree, In-degree, and Out-degree

1. Complete Binary Tree

A complete binary tree is a binary tree in which:
All levels are completely filled except possibly the last level.
The last level is filled from left to right.

Example:



Here, all levels are filled except the last, which is filled left to right.

2. In-degree

In-degree of a node = number of incoming edges.

For binary trees:

Root node → in-degree = 0
Other nodes → in-degree = 1

Example: Node 1 (root) → in-degree 0, Node 2 → in-degree 1.

3. Out-degree

Out-degree of a node = number of children a node has.

In a binary tree, maximum out-degree = 2.

Example:

Node 1 has 2 children → out-degree = 2
Node 4 has 0 children → out-degree = 0 (leaf node)

Q43. Explain Indegree and Outdegree of Binary Tree with Example

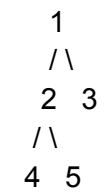
1. In-degree

Definition: Number of incoming edges to a node.

In a binary tree:

Root node → in-degree = 0
All other nodes → in-degree = 1

Example:



Node 1 (root) → in-degree = 0
Node 2 → in-degree = 1
Node 4 → in-degree = 1

2. Out-degree

Definition: Number of children a node has.
In a binary tree, maximum out-degree = 2.

Example (same tree):

Node 1 → out-degree = 2 (children: 2, 3)
Node 2 → out-degree = 2 (children: 4, 5)
Node 4 → out-degree = 0 (leaf node)

Summary Table

Node	In-degree	Out-degree
1	0	2
2	1	2
3	1	0
4	1	0
5	1	0

Q44. Define Tree, Enlist Types of Tree, and Write an Algorithm to Insert Node in Binary Search Tree (BST) Using Python

1. Definition of Tree

A tree is a hierarchical data structure consisting of nodes connected by edges.

Each tree has:

Root node: The topmost node
Child node: Nodes directly connected below a node
Parent node: Node connected above a node
Leaf node: Node with no children

2. Types of Trees

1. Binary Tree – Each node has at most two children.
2. Binary Search Tree (BST) – Left child < root < right child.
3. Complete Binary Tree – All levels filled except last, which is filled left to right.
4. Full Binary Tree – Every node has 0 or 2 children.
5. AVL Tree – Balanced BST where height difference ≤ 1.
6. N-ary Tree – Each node can have n children.
7. Red-Black Tree – Self-balancing BST with color properties.

3. Algorithm to Insert Node in BST (Python)

```
# Define Node class
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

```
# Function to insert a new node
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

# Example Usage
root = Node(50)
root = insert(root, 30)
root = insert(root, 70)
root = insert(root, 20)
root = insert(root, 40)
root = insert(root, 60)
root = insert(root, 80)
```

Q45. Write an Algorithm for Insertion and Deletion of a Node in Binary Search Tree (BST)

1. Insertion in BST

Algorithm

INSERT_BST(root, key):
1. If root is NULL:
2. Create a new node with key
3. Return the new node
4. If key < root.key:
5. root.left = INSERT_BST(root.left, key)
6. Else if key > root.key:
7. root.right = INSERT_BST(root.right, key)
8. Return root

Explanation

If the tree is empty → create a new node as root.
If key < current node → go to left subtree.
If key > current node → go to right subtree.
Recursively insert until proper position is found.

2. Deletion in BST

Algorithm

```
DELETE_BST(root, key):
1. If root is NULL:
2.     Return root
3. If key < root.key:
4.     root.left = DELETE_BST(root.left, key)
5. Else if key > root.key:
6.     root.right = DELETE_BST(root.right, key)
7. Else:
8.     # Node found
9.     If node has no child:
10.        Delete node, return NULL
11.    Else if node has one child:
12.        Replace node with child
13.    Else:
14.        Find inorder successor (minimum in right subtree)
15.        Replace node.key with successor.key
16.        Delete successor recursively
17. Return root
```

Explanation

No child: simply delete the node.
One child: replace node with its child.
Two children: replace node with inorder successor (smallest node in right subtree) and delete the successor.

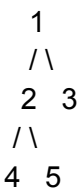
Q46. Define Path and Leaf Node

1. Path in a Tree

A path is a sequence of nodes connected by edges in a tree.

Length of path: Number of edges in the path.

Example:



Path from node 1 to 5 → 1 → 2 → 5

Path length = 2 (edges 1→2 and 2→5)

2. Leaf Node

A leaf node is a node that does not have any children.

Characteristics:

Out-degree = 0

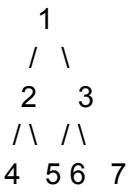
Found at the bottom of the tree

Example (same tree):

Leaf nodes: 3, 4, 5

Q47. Perform Inorder, Preorder, and Postorder Traversal of a Binary Tree

Given Binary Tree



1. Inorder Traversal (Left → Root → Right)

Steps: Visit left subtree, then root, then right subtree

Traversal Sequence: 4, 2, 5, 1, 6, 3, 7

2. Preorder Traversal (Root → Left → Right)

Steps: Visit root, then left subtree, then right subtree

Traversal Sequence: 1, 2, 4, 5, 3, 6, 7

3. Postorder Traversal (Left → Right → Root)

Steps: Visit left subtree, then right subtree, then root

Traversal Sequence: 4, 5, 2, 6, 7, 3, 1

Q48. Create a Binary Search Tree (BST) for Given Keys and Write Traversals

Given Keys:

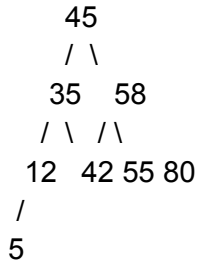
45, 35, 12, 58, 5, 55, 58, 80, 35, 42

1. Constructing the BST

Step-by-step insertion (ignoring duplicates):

1. Insert 45 → root
2. Insert 35 → left of 45
3. Insert 12 → left of 35
4. Insert 58 → right of 45
5. Insert 5 → left of 12
6. Insert 55 → left of 58
7. Insert 58 → duplicate → ignore or keep as right child
8. Insert 80 → right of 58
9. Insert 35 → duplicate → ignore
10. Insert 42 → right of 35

Final BST Structure:



2. Traversals

a) Inorder Traversal (Left → Root → Right)

Sequence: 5, 12, 35, 42, 45, 55, 58, 80

b) Preorder Traversal (Root → Left → Right)

Sequence: 45, 35, 12, 5, 42, 58, 55, 80

c) Postorder Traversal (Left → Right → Root)

Sequence: 5, 12, 42, 35, 55, 80, 58, 45